

TREEMAPPO: FINE-GRAINED CREDIT ASSIGNMENT FOR AGENTIC LLM TRAINING VIA TOKEN-LEVEL TRAJECTORY BRANCHING

Anonymous authors

Paper under double-blind review

ABSTRACT

We present TreeMAPPO, a reference-free framework for fine-grained credit assignment in agentic large language model training. Rather than assigning a single scalar reward to an entire reasoning trajectory, TreeMAPPO constructs a roll-out tree, estimates segment-level contribution from mixed-success leaves, and converts these estimates into dense token-level training signals. The method is designed for mathematical reasoning agents whose trajectories may interleave natural-language reasoning, fenced Python tool calls, tool responses, and final boxed answers. Compared with methods such as TEMPO that exploit shared prefixes among sampled trajectories, TreeMAPPO actively creates new continuations through guided token-level branching, concentrating exploration at structurally useful and model-plausible decision points. Compared with TreeRPO-style tree credit assignment based on empirical outcome comparisons, TreeMAPPO uses a statistically grounded maximum-a-posteriori posterior model that estimates signed segment contribution before producing token-level advantages. Experiments use a resource-efficient split pipeline that rolls out from the base policy, generates advantage-weighted training data, fine-tunes LoRA adapters, and validates checkpoint snapshots. Current results show consistent but modest validation gains across several model families, stronger gains in some tool-use settings, and mixed transfer to broader held-out tests.

1 INTRODUCTION

Sparse end-of-trajectory supervision makes credit assignment a central bottleneck in reinforcement learning for reasoning agents. When a long rollout contains many reasoning tokens, optional tool calls, and only a terminal correctness judgment, a single trajectory-level reward provides little guidance about which intermediate decisions should be reinforced or suppressed. The difficulty is especially acute in agentic settings, where early errors may be repaired by later reasoning or external tool feedback.

Recent work has begun to exploit shared-prefix and tree structure to densify supervision, including implicit prefix-tree methods, explicit tree-sampling methods, off-policy prefix-aware methods, and fine-grained agentic branching methods Tran et al. (2025); Yang et al. (2025); Li et al. (2025); Huang et al. (2025); Wang et al. (2026). These methods show that branching structure exposes local comparisons that are unavailable to flat rollout groups. However, existing approaches often rely on naturally occurring prefix overlap, fixed segment boundaries, offline teacher-generated trees, or heuristic credit estimators that do not explicitly represent uncertainty.

This paper develops fine-grained credit assignment for mathematical reasoning agents through token-level trajectory branching. The central idea is to turn a small set of full rollouts into a structured tree of alternative continuations, then infer which segments are helpful, harmful, or uncertain from the pattern of correct and incorrect descendants. We call the resulting method TreeMAPPO (Tree-based Maximum-A-Posteriori Policy Optimization).

Our implementation uses a lightweight tool-calling protocol: the model receives a question and an optional Python tool specification, emits reasoning in text, may invoke the tool via fenced Python markdown blocks, observes the tool response in context, and is required to terminate with a fi-

nal answer wrapped in `\boxed{}`. Rollouts are organized into trees, branch points are chosen at token granularity, and segment-level posterior estimates are transformed into token-level training advantages. The experiments in this submission emphasize a split one-shot pipeline: collect rollouts from the base model, construct the training set once for each condition, train adapters for multiple epochs, and validate the resulting checkpoints. This isolates the proposed credit-assignment mechanism while keeping the compute budget tractable.

Our contributions are:

- We formulate fine-grained credit assignment as posterior inference over signed segment contributions in a trajectory tree.
- We introduce a guided branching policy that prioritizes model-plausible token-level alternatives while discouraging over-fragmentation and over-concentration, with forced first-token branching used to make the intended alternative explicit.
- We describe a practical training pipeline that converts rollout trees into non-redundant training trajectories with token-level advantages.
- We report split-pipeline LoRA experiments showing small positive validation gains in Qwen2.5-7B and Qwen3-4B settings, together with negative results that identify instability at overly aggressive learning rates and mixed held-out transfer.

2 RELATED WORK

Trajectory-level RL for reasoning. Reinforcement learning with verifiable rewards has become a standard recipe for improving mathematical and agentic reasoning, because final-answer correctness can often be checked automatically. PPO remains a canonical policy-optimization baseline: it stabilizes policy updates with a clipped surrogate objective and obtains token-level advantages through a learned value function Schulman et al. (2017). GRPO, introduced in DeepSeekMath, removes the critic by normalizing outcome rewards within groups of sampled responses, substantially reducing training complexity while retaining the appeal of verifiable rewards Shao et al. (2024). The limitation of both trajectory-level and group-relative outcome supervision is the one targeted here: when a long response receives only terminal feedback, many irrelevant tokens are reinforced or suppressed together with the decisive reasoning steps.

Process supervision and verifier-based credit. A complementary line of work makes supervision denser by judging intermediate reasoning steps. Let’s Verify Step by Step showed that process supervision can outperform outcome-only supervision on mathematical reasoning by training process reward models from step-level human labels Lightman et al. (2023). This line establishes the value of fine-grained credit, but it changes the supervision regime: it depends on annotated intermediate labels, learned verifiers, or additional reward-model infrastructure. TreeMAPPO instead keeps the outcome-only setting and infers local credit from the structure of sampled rollouts. The distinction is important methodologically: our goal is not to build a stronger verifier, but to convert sparse terminal correctness into localized training signals without step-level references.

Implicit tree and shared-prefix methods. TEMPO Tran et al. (2025) converts a group of sampled responses into a prefix tree and adds branch-gated temporal-difference corrections on top of a GRPO-style training loop. Its main appeal is that it preserves a relatively simple rollout interface while extracting token-level signal where natural branching occurs. Our method differs in that we do not rely on incidental prefix overlap alone; instead, we explicitly create new branches at selected token positions, which provides direct control over branching density using rollout structure and model-side alternative-token probabilities.

Explicit tree sampling and tree-aware advantages. TreeRPO Yang et al. (2025) and TreePO Li et al. (2025) both pursue explicit tree-structured sampling to obtain denser supervision. TreeRPO estimates step-level reward expectations via tree sampling and forms step-level comparison groups without a separate reward model. TreePO emphasizes systems efficiency, shared-prefix reuse, and subgroup-relative advantages over segment-level trees. Our method is closest in spirit to this line,

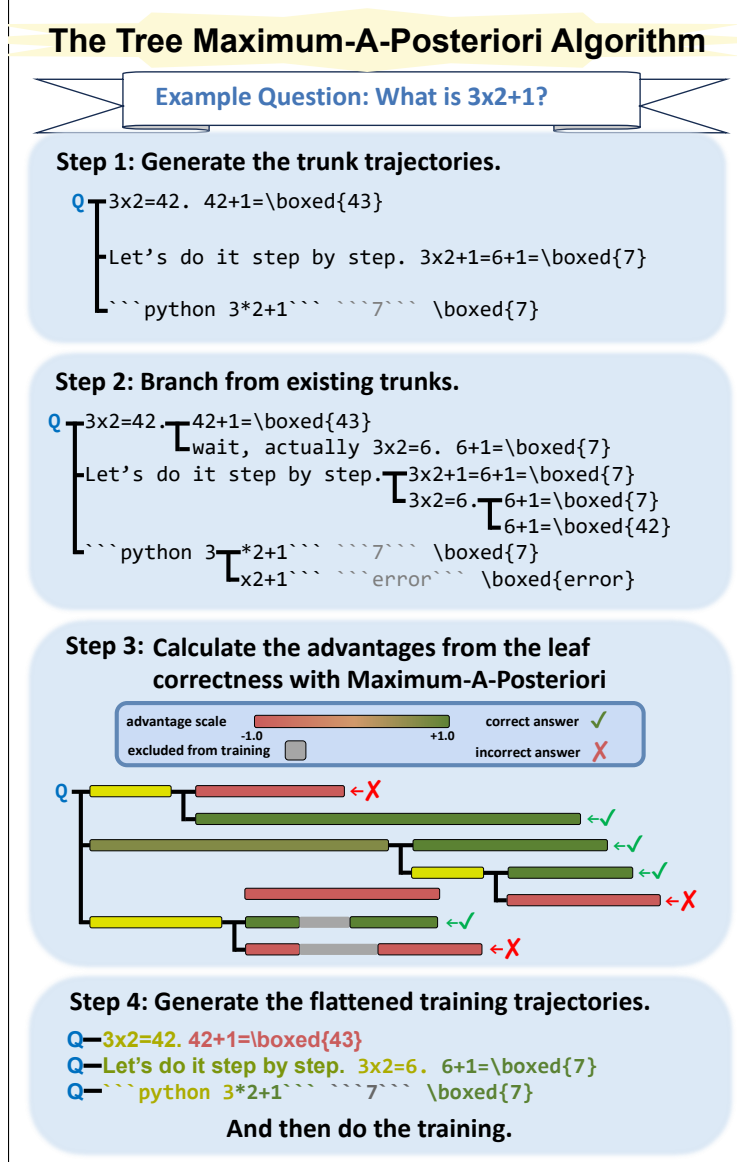


Figure 1: Method overview. TreeMAPPO samples base-policy trajectories, expands selected token-level branch points, infers segment-level contribution and uncertainty from leaf correctness, and converts the resulting posterior estimates into token-level advantages for adapter training.

but differs in two important respects: first, we branch at arbitrary token positions rather than imposing fixed step or segment partitions; second, we infer segment contribution with a probabilistic MAP model that explicitly represents uncertainty, rather than relying only on win-rate or subgroup heuristics.

Prefix-aware and off-policy tree supervision. Tree-OPO Huang et al. (2025) uses teacher-generated MCTS prefixes and staged advantage estimation to create prefix-aware off-policy training groups. By contrast, TreeMAPPO is on-policy and reference-free: the tree is constructed directly from the policy’s own rollouts, without offline teacher trees, reward models, or process-level supervision.

Agentic RL and branching granularity. APPO Wang et al. (2026) studies branching and credit assignment in tool-using agents, arguing that influential decision points are not confined to tool-

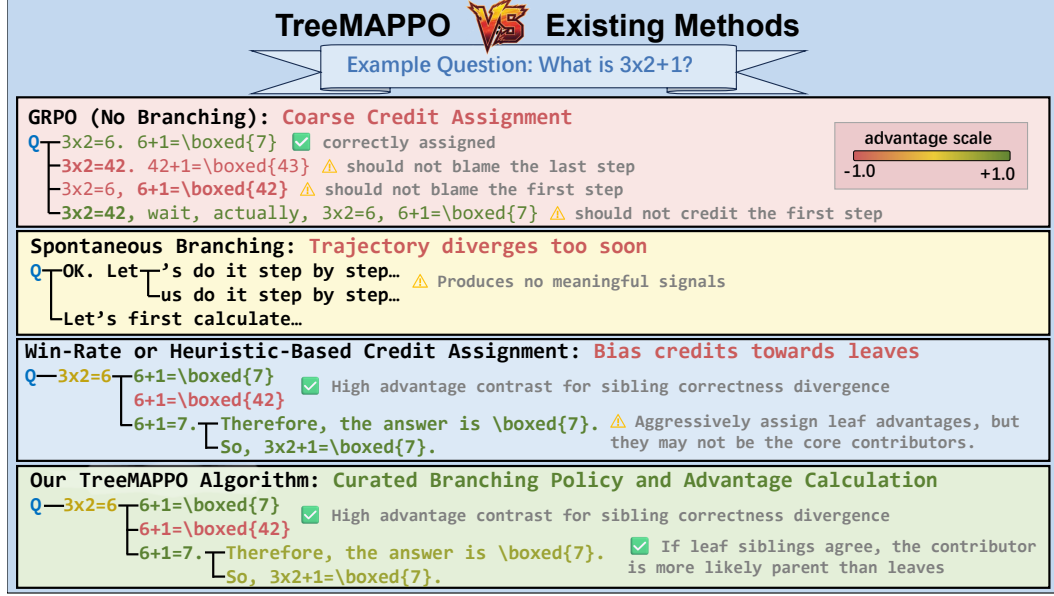


Figure 2: Comparison of credit-assignment structures. Trajectory-level GRPO assigns one normalized outcome signal to every token; TEMPO extracts branch-gated corrections from natural shared prefixes; TreeRPO uses fixed-step tree sampling and empirical child-group rewards; TreeMAPPO combines guided token-level branching with posterior segment-credit inference.

call boundaries or coarse workflow stages. This observation aligns closely with our setting. The main methodological difference is that APPO uses a branching score and procedure-level advantage scaling, whereas TreeMAPPO combines explicit token-level branching with posterior inference over signed segment contribution and uncertainty.

Tool-augmented mathematical reasoning and benchmarks. Our agent protocol is also related to work on tool-augmented reasoning. PAL and Program-of-Thoughts showed that language models can delegate arithmetic or symbolic computation to Python programs, improving numerical reasoning by separating decomposition from execution Gao et al. (2022); Chen et al. (2022). ReAct generalized interleaved reasoning and acting, while Toolformer studied how models can learn tool use from limited demonstrations Yao et al. (2022); Schick et al. (2023). These methods motivate the mixed text–tool trajectories considered here, but they primarily address inference-time tool use rather than training-time credit assignment. Our evaluation setting builds on mathematical reasoning resources such as MATH Hendrycks et al. (2021) and recent verifiable training corpora such as DeepMath-103K He et al. (2025); the broader benchmark mix is intended to test whether tree-based credit assignment transfers beyond the distribution used to construct training trajectories.

3 PROBLEM SETTING AND AGENT PROTOCOL

We study question-answering agents for mathematical reasoning. Each sample provides a problem statement and a ground-truth final answer used only for post hoc correctness judgment. During rollout, the model may produce plain-text reasoning and, when enabled, invoke a Python executor.

Our implementation uses the following protocol.

1. Each prompt contains the problem statement and, when tool use is enabled, a specification that Python can be called by emitting a fenced markdown block.
2. The model may interleave reasoning and tool use.
3. When a fenced Python block closes, generation pauses, the tool executes, and the tool response is appended to the context.
4. The model is instructed to present its final answer in $\boxed{\{ \}}$.

5. If generation reaches a control handoff without a boxed answer, the framework requests continuation until a boxed answer is produced or rollout termination conditions are hit.

This protocol is consistent across training-data collection and evaluation so that differences across methods are attributable to branching and credit assignment rather than prompt format changes.

4 METHOD

4.1 OVERVIEW

For each question, we first sample a set of complete trunk trajectories. We then expand a rollout tree by branching from selected token positions. Each completed leaf trajectory is judged as correct or incorrect using its extracted final boxed answer. The resulting leaf labels provide supervision for segment-level posterior inference. Posterior estimates are transformed into branching scores for future expansions and into training advantages for model optimization.

4.2 TRAJECTORY TREE CONSTRUCTION

In the default configuration for our main tool-using setup, we begin with 4 trunk trajectories and expand to 16 total trajectories. Our experimental plan also includes branch-budget variants with 8 and 32 total trajectories.

Branching is token-level rather than step-level. A new branch can be created in two ways:

- **Segment split:** split an existing reasoning segment at an internal token position and regenerate from that point.
- **Node expansion:** branch again from an existing branching node, increasing its branching factor.

The implementation keeps a fixed decoding temperature within each tree. In the current configs, this value is $T = 0.7$ for trunk generation, continuation, and branch expansion.

4.3 MAP CREDIT ASSIGNMENT MODEL

Let ℓ index judged leaves and let i index non-prompt segments. For each leaf ℓ :

- $y_\ell \in \{+1, -1\}$ is the correctness label.
- $x_{\ell,i} \in \{0, 1\}$ indicates whether segment i lies on the path to leaf ℓ .
- Segment i has mean contribution m_i and log standard deviation u_i .

Leaf-level moments are

$$\mu_\ell = \sum_i x_{\ell,i} m_i, \quad (1)$$

$$\tau_\ell = \sqrt{\sum_i x_{\ell,i} \exp(2u_i) + \varepsilon}, \quad (2)$$

$$z_\ell = y_\ell \mu_\ell / \tau_\ell. \quad (3)$$

We use a probit likelihood,

$$p(y_\ell \mid \{m_i, u_i\}) = \Phi(z_\ell), \quad (4)$$

where Φ is the standard normal CDF.

Our implementation initializes prior means at zero and optimizes the negative log posterior

$$\mathcal{J} = - \sum_\ell \log \Phi(z_\ell) + \frac{1}{2\sigma_m^2} \sum_i m_i^2 + \frac{1}{2\sigma_u^2} \sum_i u_i^2. \quad (5)$$

The current hyperparameter file uses $\sigma_m = 1.0$, $\sigma_u = 1.0$, $\varepsilon = 10^{-6}$, 120 optimization iterations, and log-standard-deviation clamps in $[-4, 2]$. Optimization is performed with gradient steps

and backtracking line search, together with a numerically stable implementation of $\log \Phi$ for large negative margins.

4.4 GUIDED TOKEN-LEVEL BRANCHING

The current implementation chooses branch points before leaf judgments are available. Branching therefore relies on rollout structure and model-side token statistics rather than posterior uncertainty. For each candidate token position, the implementation multiplies:

1. a structural penalty, and
2. the relative probability of the best alternative token that differs from the already-sampled continuation.

Candidates whose best alternative has relative probability below 0.1 are discarded.

For node expansion, the structural penalty is an exponential branching-factor penalty,

$$\gamma_{\text{node}}(b) = \exp(-k_{\text{node}}(b-1)), \quad (6)$$

with $k_{\text{node}} = 0.35$ in the current implementation. For segment splitting, the structural penalty increases with the shorter side of the split relative to average trunk length, discouraging very short fragments.

After a branch point and alternative token are selected, the current guided-branching implementation forces that selected token to be the first token of the new branch continuation. This makes the token-level branch explicit rather than merely restarting generation from the shared prefix and hoping that sampling chooses the intended alternative. We keep the non-forced version as an ablation, but use the forced-token version as the default because it better matches the intended token-level intervention.

4.5 FROM POSTERIOR SCORES TO TRAINING ADVANTAGES

After tree construction, each segment receives an unnormalized signed score proportional to $m_i / \exp(u_i)$. The methodology notes propose an additional length-aware rescaling to reduce volatility from very short reasoning spans. The generated training samples then attach token-level advantage values aligned with the input sequence.

The downstream training code consumes aligned token sequences, supervised target tokens, attention masks, token-level advantages, and old token log probabilities. The loss is a clipped policy-ratio surrogate over supervised positions,

$$\mathcal{L}_{\text{train}} = -\frac{1}{|\mathcal{S}|} \sum_{t \in \mathcal{S}} \min(\rho_t \tilde{A}_t, \text{clip}(\rho_t, 1 - \epsilon, 1 + \epsilon) \tilde{A}_t), \quad (7)$$

where $\mathcal{S}$ denotes supervised token positions, ρ_t is the ratio between the current token probability and the stored old token probability, and $\tilde{A}_t$ is the token advantage clipped to $[-3, 3]$. This is the implemented optimization interface for injecting segment-level credit into model updates while avoiding the unbounded negative-advantage objective that arises from direct advantage-weighted cross entropy.

4.6 TRAINING-SET CONSTRUCTION

Because many tree leaves share prefixes, naively flattening all leaves would over-train shared segments. We therefore use a greedy de-duplication policy.

1. Score each candidate leaf trajectory by average absolute token advantage.
2. Select the strongest candidate.
3. Mark its segments as consumed.
4. Repeat among trajectories that still contain unconsumed learnable segments.
5. Aggregate selected trajectories across trees and prune the tail with a cumulative average-absolute-advantage cutoff.

Selected trajectories are finally ordered by a configurable batching policy. In length-based runs, this improves early memory stability; in by-question runs, it groups positive and negative samples from the same problem so the optimizer sees local contrasts consecutively.

5 BASELINES AND ABLATIONS

We consider three primary comparison points.

- **Base model:** no fine-tuning.
- **GRPO-style baseline:** use 8 trunks and 8 total trajectories, yielding multiple complete rollouts without token-level branching in the main tool setup.
- **TreeMAPPO:** guided branching with MAP-based posterior credit assignment.

We also separate two ablation axes that are explicitly encoded in configs and scripts.

- **TEMPO-style branching ablation:** switch the branching policy from guided branching to spontaneous divergence-point branching while keeping posterior-based credit assignment.
- **TreeRPO-style credit ablation:** keep guided branching but replace posterior-based segment advantages with bottom-up child-group outcome comparisons. For each branching group, terminal correctness is backed up into child subtree values; groups with negligible value spread are filtered; surviving children receive normalized relative advantages.
- **TreeRL-style branching ablation:** keep posterior-based credit assignment but select guided branch points by model entropy rather than our full branching score.
- **TreeRL-style credit ablation:** keep guided branching but replace posterior-based segment advantages with a local-global value-difference estimator, combining a segment’s improvement over its parent with its improvement over the root.

In our implementation, these ablations are not merely renamed conditions: the TEMPO and TreeRL-branching ablations change the exploration rule, while the TreeRPO-style and TreeRL-credit ablations change the advantage-estimation rule. This distinction is important because it cleanly separates exploration structure from credit estimation.

6 EXPERIMENTAL SETUP

6.1 MODELS

We evaluate the following model-condition families.

- Qwen2.5-7B-Instruct with tool use.
- Qwen2.5-7B-Instruct without tool use.
- Qwen3-4B with tool use.
- Qwen3-4B without tool use.
- Gemma-3-4B-IT without tool use.
- Llama-3.1 and Mistral-7B without tool use.

For the main tables, tool-enabled and no-tool variants of the same backbone are treated as separate model-condition groups, since the prompting interface and action space differ.

6.2 DATASETS

Training and validation are built from DeepMath, MATH, and NuminaMath.

- The training JSONL file contains 5,000 samples per dataset, interleaved across the three in-distribution datasets.

- The validation JSONL file contains 1,000 validation samples per dataset drawn from disjoint index ranges after the training slice.

We also construct an aggregated held-out evaluation JSONL file from six datasets in this order: DeepMath, MATH, NuminaMath, AMC2023, GaoKao Math 2024, and CollegeMath. For datasets with more than 1,000 test rows, 1,000 are sampled with seed 42; for DeepMath, which lacks a dedicated test split in the current pipeline, evaluation rows are drawn from the training split after clearing the reserved training and validation examples.

6.3 TRAINING AND EVALUATION PROTOCOL

The primary experiments use a split one-shot protocol rather than a fully interleaved online reinforcement-learning loop. For each model-condition pair, we run four stages: base-policy rollout collection, training-set generation, adapter training, and checkpoint validation. Rollout and generation are performed once per condition using the base model, after which the generated trajectories and token-level advantages are reused across multiple LoRA training epochs. Validation then evaluates the base model and each trained checkpoint under the same inference protocol.

This protocol is intentionally different from a production online RL pipeline in which each epoch rolls out from the latest policy and immediately regenerates training data. The split protocol is less expensive, makes infrastructure failures easier to diagnose, and allows fair controlled comparisons among GRPO-style, TreeRPO-style, TEMPO-style, and TreeMAPPO variants under the same rollout budget. When compute permits, the same codebase also supports a full orchestrator mode with repeated validation rollout, training rollout collection, training-set generation, and training across policy updates; those runs are treated as confirmatory rather than the main experimental path.

The training protocol uses the following recurring settings:

- chunked one-shot training, where each epoch consumes one generated training chunk and recovery restarts at whole-chunk boundaries;
- LoRA rank 32 and learning rate 10^{-6} for the main stable runs, with Adam state and learning-rate warmup enabled unless stated otherwise;
- held-out validation at epoch 0 and then every 10 epochs for long runs, with multiple validation rollout trials used to reduce sampling noise;
- LoRA-based fine-tuning for the primary experiments, while full-model training is deferred because queueing and memory costs are substantially higher.
- fixed rollout temperature 0.7.

The primary evaluation metric is pass@1 accuracy. Validation reports an equal-weight macro average over the in-distribution validation datasets. Serious testing uses a broader six-dataset mixture and reports an equal-weight macro average over datasets rather than over examples, so smaller datasets do not vanish in the aggregate.

7 RESULTS

This section reports the audited experiment inventory available at submission time. All serious-test rows in the main tables use five rollout trials and all six test datasets; held-out validation rows use six trials. We emphasize conservative conclusions: the current evidence supports modest and repeatable validation improvements, but the broader serious tests show that these gains do not always transfer uniformly.

7.1 VALIDATION ACCURACY TRENDS

Table 1 summarizes audited held-out validation runs. “Best” is the best validated trained checkpoint, and gains are measured against the same run’s base checkpoint. All listed training runs use single-GPU LoRA rank 32, learning rate 10^{-6} , Adam, and warmup.

Across validation runs, both GRPO and TreeMAPPO extract usable signal from the split pipeline. Qwen2.5-7B gains are modest in both no-tool and tool settings, roughly one percentage point at the

Model and condition	Method	Trials	Epochs audited	Base	Best epoch	Best	Gain	Last
Qwen2.5-7B no-tool	GRPO	6	0,10,...,70	<u>0.6468</u> ± 0.0033	30	<u>0.6538</u> ± 0.0021	+0.0069	<u>0.6522</u> ± 0.0039
Qwen2.5-7B no-tool	TreeMAPPO	6	0,10,...,70	0.6436 ± 0.0025	50	0.6523 ± 0.0016	<u>+0.0087</u>	0.6494 ± 0.0035
Qwen2.5-7B + tool	GRPO	6	0,10,...,70	<u>0.6142</u> ± 0.0033	30	<u>0.6246</u> ± 0.0024	<u>+0.0104</u>	0.6171 ± 0.0046
Qwen2.5-7B + tool	TreeMAPPO	6	0,10,...,70	0.6134 ± 0.0034	50	0.6236 ± 0.0033	+0.0101	<u>0.6181</u> ± 0.0016
Qwen3-4B no-tool	GRPO	6	0,10,...,50	<u>0.6901</u> ± 0.0021	50	<u>0.6991</u> ± 0.0038	<u>+0.0089</u>	<u>0.6991</u> ± 0.0038
Qwen3-4B no-tool	TreeMAPPO	6	0,10,...,50	0.6894 ± 0.0017	30	0.6944 ± 0.0024	+0.0050	0.6849 ± 0.0047
Qwen3-4B + tool	GRPO	6	0,10,...,50	<u>0.6446</u> ± 0.0030	40	<u>0.6617</u> ± 0.0046	<u>+0.0172</u>	0.6521 ± 0.0044
Qwen3-4B + tool	TreeMAPPO	6	0,10,...,50	0.6434 ± 0.0034	50	0.6605 ± 0.0023	+0.0171	<u>0.6605</u> ± 0.0023
Gemma-3-4B no-tool	GRPO	6	0,10,...,50	<u>0.5640</u> ± 0.0022	50	<u>0.5647</u> ± 0.0032	+0.0007	<u>0.5647</u> ± 0.0032
Gemma-3-4B no-tool	TreeMAPPO	6	0,10,...,50	0.5621 ± 0.0030	40	0.5643 ± 0.0029	<u>+0.0023</u>	0.5594 ± 0.0034
Mistral-7B no-tool	GRPO	6	0,10,...,50	<u>0.1747</u> ± 0.0043	40	0.1802 ± 0.0013	+0.0055	0.1729 ± 0.0021
Mistral-7B no-tool	TreeMAPPO	6	0,10,...,50	0.1747 ± 0.0019	10	<u>0.1819</u> ± 0.0031	<u>+0.0072</u>	<u>0.1788</u> ± 0.0023
Llama-3.1 no-tool	GRPO	6	0,10,...,50	0.4415 ± 0.0038	30	<u>0.4684</u> ± 0.0018	<u>+0.0269</u>	<u>0.4609</u> ± 0.0064
Llama-3.1 no-tool	TreeMAPPO	6	0,10,...,50	<u>0.4454</u> ± 0.0035	40	0.4589 ± 0.0029	+0.0135	0.4568 ± 0.0053

Table 1: Audited held-out validation results. Values are pass@1 macro averages on the validation mixture, reported as mean with 95% confidence interval over rollout trials. All rows use six validation trials and epochs that are 0 or multiples of 10. Bold underlined entries mark the largest value within each model-condition comparison block.

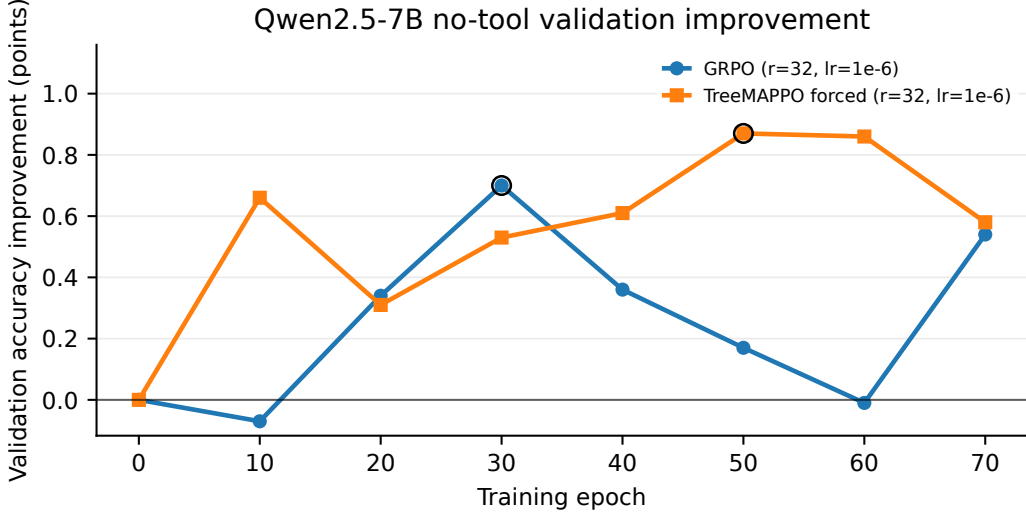


Figure 3: Validation accuracy improvement over the base checkpoint for Qwen2.5-7B in the no-tool setting. Both lines use LoRA rank 32 and learning rate 10^{-6} ; the y-axis reports percentage-point change relative to each method’s epoch-0 validation accuracy.

best audited checkpoints. Qwen3-4B shows stronger tool-use gains for both methods, and Llama-3.1 improves under GRPO before partially regressing. Gemma and Mistral remain weak in absolute accuracy, but their best checkpoints still show small positive validation movement. The main pattern is therefore not monotonic scaling with epoch count; checkpoint selection by validation is necessary.

7.2 HELD-OUT SERIOUS TESTS

Table 2 reports serious tests on the broader six-dataset mixture. Each row uses the best checkpoint selected by validation for that method and setting, then evaluates it with five rollout trials when the repaired per-trial artifacts are available. The macro average gives equal weight to datasets rather than to individual examples. Checkpoint selection and artifact coverage are reported separately in Table 3.

The serious tests sharpen the conclusion. In no-tool Qwen2.5-7B, neither trained method beats the base model on the six-dataset macro average despite validation gains, indicating limited transfer from the validation mixture. In tool-enabled Qwen2.5-7B, both trained methods improve over base by about one point, with TreeMAPPO close to GRPO. Qwen3-4B is the strongest positive setting: both methods improve in no-tool testing, and TreeMAPPO substantially outperforms GRPO in the

Model and condition	Method	In-distribution datasets			Out-of-distribution datasets			
		DeepMath	MATH	NuminaMath	AMC 2023	GaoKao 2024	CollegeMath	Macro avg.
Qwen2.5-7B no-tool	Base	0.6034 $\pm$ 0.0030	0.7400$\pm$0.0056	0.6160$\pm$0.0211	0.5400 $\pm$ 0.0480	0.6000 $\pm$ 0.0511	0.7204$\pm$0.0029	0.6366$\pm$0.0145
Qwen2.5-7B no-tool	GRPO	0.5886 $\pm$ 0.0028	0.7338 $\pm$ 0.0056	0.5940 $\pm$ 0.0133	0.5200 $\pm$ 0.0475	0.6692$\pm$0.0185	0.7062 $\pm$ 0.0042	0.6353 $\pm$ 0.0118
Qwen2.5-7B no-tool	TreeMAPPO	0.6050$\pm$0.0059	0.7362 $\pm$ 0.0031	0.5880 $\pm$ 0.0073	0.5900$\pm$0.0480	0.5615 $\pm$ 0.0185	0.7016 $\pm$ 0.0043	0.6304 $\pm$ 0.0081
Qwen2.5-7B + tool	Base	0.5806 $\pm$ 0.0042	0.6812 $\pm$ 0.0042	0.5540 $\pm$ 0.0048	0.4650 $\pm$ 0.0720	0.5154 $\pm$ 0.0511	0.6488 $\pm$ 0.0082	0.5742 $\pm$ 0.0118
Qwen2.5-7B + tool	GRPO	0.5806 $\pm$ 0.0063	0.6988 $\pm$ 0.0053	0.5660 $\pm$ 0.0100	0.4750$\pm$0.0410	0.5385$\pm$0.0413	0.6514 $\pm$ 0.0041	0.5850$\pm$0.0149
Qwen2.5-7B + tool	TreeMAPPO	0.5840$\pm$0.0060	0.7036$\pm$0.0046	0.5780$\pm$0.0243	0.4600 $\pm$ 0.0504	0.5231 $\pm$ 0.0185	0.6530$\pm$0.0028	0.5836 $\pm$ 0.0116
Qwen3-4B no-tool	Base	0.6716 $\pm$ 0.0035	0.8244 $\pm$ 0.0032	0.6240 $\pm$ 0.0202	0.5800 $\pm$ 0.0392	0.7615 $\pm$ 0.0282	0.7022 $\pm$ 0.0022	0.6940 $\pm$ 0.0109
Qwen3-4B no-tool	GRPO	0.6846$\pm$0.0038	0.8250 $\pm$ 0.0069	0.6340$\pm$0.0078	0.6250$\pm$0.0219	0.7231 $\pm$ 0.0369	0.7078$\pm$0.0042	0.6999$\pm$0.0037
Qwen3-4B no-tool	TreeMAPPO	0.6840 $\pm$ 0.0092	0.8368$\pm$0.0047	0.6120 $\pm$ 0.0073	0.5850 $\pm$ 0.0250	0.7769$\pm$0.0554	0.7034 $\pm$ 0.0022	0.6997 $\pm$ 0.0098
Qwen3-4B + tool	Base	0.6078 $\pm$ 0.0056	0.7458 $\pm$ 0.0044	0.5900 $\pm$ 0.0164	0.5850 $\pm$ 0.0250	0.5692 $\pm$ 0.0440	0.6616 $\pm$ 0.0029	0.6266 $\pm$ 0.0099
Qwen3-4B + tool	GRPO	0.6288 $\pm$ 0.0063	0.7734$\pm$0.0073	0.5480 $\pm$ 0.0144	0.6650$\pm$0.0250	0.6077 $\pm$ 0.0369	0.6692 $\pm$ 0.0069	0.6487 $\pm$ 0.0084
Qwen3-4B + tool	TreeMAPPO	0.6340$\pm$0.0095	0.7670 $\pm$ 0.0021	0.6020$\pm$0.0096	0.6500 $\pm$ 0.0410	0.6846$\pm$0.0282	0.6788$\pm$0.0035	0.6694$\pm$0.0101
Gemma-3-4B no-tool	Base	0.4780 $\pm$ 0.0067	0.7606 $\pm$ 0.0043	0.5580 $\pm$ 0.0114	0.4800 $\pm$ 0.0646	0.5692 $\pm$ 0.0282	0.6456$\pm$0.0049	0.5819 $\pm$ 0.0115
Gemma-3-4B no-tool	GRPO	0.4810 $\pm$ 0.0059	0.7610 $\pm$ 0.0027	0.5920$\pm$0.0114	0.4300 $\pm$ 0.0240	0.5769 $\pm$ 0.0238	0.6402 $\pm$ 0.0039	0.5802 $\pm$ 0.0093
Gemma-3-4B no-tool	TreeMAPPO	0.4846$\pm$0.0122	0.7670$\pm$0.0078	0.5720 $\pm$ 0.0130	0.5050$\pm$0.0567	0.5769$\pm$0.0238	0.6382 $\pm$ 0.0047	0.5906$\pm$0.0117
Mistral-7B no-tool	Base	0.1910 $\pm$ 0.0092	0.1474 $\pm$ 0.0035	0.1900 $\pm$ 0.0124	0.0650$\pm$0.0332	0.2154$\pm$0.0452	0.1728 $\pm$ 0.0040	0.1636 $\pm$ 0.0129
Mistral-7B no-tool	GRPO	0.2136$\pm$0.0062	0.1528 $\pm$ 0.0063	0.2560$\pm$0.0237	0.0500 $\pm$ 0.0155	0.1538 $\pm$ 0.0238	0.1782$\pm$0.0069	0.1674$\pm$0.0031
Mistral-7B no-tool	TreeMAPPO	0.2014 $\pm$ 0.0058	0.1562$\pm$0.0061	0.1980 $\pm$ 0.0180	0.0350 $\pm$ 0.0250	0.1769 $\pm$ 0.0511	0.1728 $\pm$ 0.0032	0.1567 $\pm$ 0.0097
Llama-3.1 no-tool	Base	0.3250 $\pm$ 0.0065	0.4954 $\pm$ 0.0032	0.4040 $\pm$ 0.0100	0.2850 $\pm$ 0.0332	0.3846 $\pm$ 0.0238	0.4642 $\pm$ 0.0052	0.3930 $\pm$ 0.0036
Llama-3.1 no-tool	GRPO	0.3666$\pm$0.0111	0.5168$\pm$0.0071	0.4340 $\pm$ 0.0159	0.2750 $\pm$ 0.0268	0.4077$\pm$0.0452	0.4688 $\pm$ 0.0051	0.4115 $\pm$ 0.0086
Llama-3.1 no-tool	TreeMAPPO	0.3314 $\pm$ 0.0045	0.5146 $\pm$ 0.0118	0.4480$\pm$0.0114	0.3200$\pm$0.0098	0.4077$\pm$0.0612	0.4784$\pm$0.0080	0.4167$\pm$0.0081

Table 2: Held-out serious-test results by dataset. Values are pass@1 accuracies with 95% confidence intervals from per-trial score artifacts. Bold underlined entries mark the largest value within each model-condition comparison block. Table 4 separately compares forced and non-forced TreeMAPPO variants.

Model and condition	Method	Selected epoch	Testing trials	Dataset coverage
Qwen2.5-7B no-tool	Base	0	5	6/6
Qwen2.5-7B no-tool	GRPO	30	5	6/6
Qwen2.5-7B no-tool	TreeMAPPO	50	5	6/6
Qwen2.5-7B + tool	Base	0	5	6/6
Qwen2.5-7B + tool	GRPO	30	5	6/6
Qwen2.5-7B + tool	TreeMAPPO	50	5	6/6
Qwen3-4B no-tool	Base	0	5	6/6
Qwen3-4B no-tool	GRPO	50	5	6/6
Qwen3-4B no-tool	TreeMAPPO	30	5	6/6
Qwen3-4B + tool	Base	0	5	6/6
Qwen3-4B + tool	GRPO	40	5	6/6
Qwen3-4B + tool	TreeMAPPO	50	5	6/6
Gemma-3-4B no-tool	Base	0	5	6/6
Gemma-3-4B no-tool	GRPO	50	5	6/6
Gemma-3-4B no-tool	TreeMAPPO	40	5	6/6
Mistral-7B no-tool	Base	0	5	6/6
Mistral-7B no-tool	GRPO	40	5	6/6
Mistral-7B no-tool	TreeMAPPO	20	5	6/6
Llama-3.1 no-tool	Base	0	5	6/6
Llama-3.1 no-tool	GRPO	30	5	6/6
Llama-3.1 no-tool	TreeMAPPO	40	5	6/6

Table 3: Checkpoint-selection and artifact-coverage table for the serious tests in Table 2. All listed rows use repaired per-trial, per-dataset artifacts under the current judging/scoring path.

tool setting. Gemma and Llama provide additional positive evidence for TreeMAPPO over their base models, while Mistral remains unstable and below base for TreeMAPPO.

7.3 ABLATION STUDY

The main ablation plan separates branching policy from credit estimation. The TEMPO-style condition changes the branch source to spontaneous shared-prefix divergence. The TreeRPO-style condition changes only the segment-credit estimator to child-group win-rate comparison. The two TreeRL-style conditions separately test entropy-guided branching and local-global value-difference

Condition	Variant	Best val. epoch	Best val.	Test epoch	Test macro
Qwen2.5-7B no-tool	Non-forced token	20	0.6472 $\pm$ 0.0018	20	0.6294 $\pm$ 0.0085
Qwen2.5-7B no-tool	Forced token	50	0.6523$\pm$0.0016	50	0.6304$\pm$0.0081
Qwen2.5-7B + tool	Non-forced token	70	0.6193 $\pm$ 0.0047	70	0.5962$\pm$0.0044
Qwen2.5-7B + tool	Forced token	50	0.6236$\pm$0.0033	50	0.5836 $\pm$ 0.0116

Table 4: Forced-token mechanism ablation for TreeMAPPO. Forced-token branching improves the selected validation checkpoint in both Qwen2.5-7B settings, but the held-out serious-test difference is small and reverses in the tool setting.

Qwen2.5-7B no-tool variant	Best val. epoch	Val. base	Best val.	Val. gain	Test epoch	Test macro
GRPO baseline	30	0.6468 $\pm$ 0.0033	0.6538$\pm$0.0021	+0.0069	30	0.6353$\pm$0.0118
TreeMAPPO, non-forced token	20	0.6399 $\pm$ 0.0026	0.6472 $\pm$ 0.0018	+0.0072	20	0.6294 $\pm$ 0.0085
TreeMAPPO, forced token	50	0.6436 $\pm$ 0.0025	0.6523 $\pm$ 0.0016	+0.0087	50	0.6304 $\pm$ 0.0081
TEMPO-style branching	70	0.6418 $\pm$ 0.0032	0.6515 $\pm$ 0.0020	+0.0097	70	0.6326 $\pm$ 0.0144
TreeRPO-style credit	40	0.6426 $\pm$ 0.0027	0.6519 $\pm$ 0.0018	+0.0094	40	0.6272 $\pm$ 0.0054
TreeRL-style branching	20	0.6402 $\pm$ 0.0045	0.6480 $\pm$ 0.0041	+0.0078	20	0.6195 $\pm$ 0.0069
TreeRL-style credit	60	0.6474$\pm$0.0044	0.6528 $\pm$ 0.0014	+0.0054	60	0.6334 $\pm$ 0.0062
TreeRL combined	40	0.6449 $\pm$ 0.0032	0.6507 $\pm$ 0.0024	+0.0058	40	0.6341 $\pm$ 0.0110
Branch budget 8	10	0.6446 $\pm$ 0.0040	0.6471 $\pm$ 0.0031	+0.0025	10	0.6274 $\pm$ 0.0076
Branch budget 32	60	0.6428 $\pm$ 0.0026	0.6503 $\pm$ 0.0041	+0.0075	60	0.6333 $\pm$ 0.0105

Table 5: Qwen2.5-7B no-tool ablations. Validation numbers are held-out macro averages with 95% confidence intervals from six rollout trials; testing uses five trials and six datasets. Gain is the difference between the best validation checkpoint and the same run’s epoch-0 validation score. Bold underlined entries mark the largest value in each numeric column.

credit. Guided-branching experiments use forced selected branch tokens by default; non-forced branching is retained only as a mechanism ablation.

The ablations do not isolate a single universally dominant ingredient. TEMPO-style branching gives the largest validation gain among the ablations but does not beat GRPO or the strongest TreeRL-credit variants on serious testing. Forced branch tokens improve validation relative to the non-forced mechanism, which supports matching the sampled branch to the token probability used in the branch score, but the serious-test difference remains small. Branch budget also has no monotonic effect: the 32-trajectory variant beats the 8-trajectory variant on testing, but neither produces a large margin.

7.4 STABILITY DIAGNOSTICS

Several completed runs are negative results that shape the current claims. Qwen2.5-7B TreeMAPPO at LoRA rank 32 with learning rate 5×10^{-5} collapsed in earlier validation, while the same rank with learning rate 10^{-6} remained stable. SGD/no-warmup variants did not show useful trends and were removed from the main schedule. Mistral remains difficult under the current recipe: its absolute accuracy is low, and TreeMAPPO is below the base model on serious testing despite a small validation gain. These outcomes motivate the conservative optimizer settings used in Table 1 and support presenting adapter-rank, learning-rate, and optimizer sensitivity as central empirical findings rather than implementation details.

7.5 BRANCHING-BUDGET SENSITIVITY

Our experimental plan includes 8-, 16-, and 32-trajectory variants for the Qwen2.5-7B no-tool setting. The current serious tests do not show a simple monotonic scaling law. Branch budget 32 has a higher serious-test macro average than branch budget 8, but the gap is small and remains within the scale of other run-to-run differences.

8 DISCUSSION

The main conceptual benefit of TreeMAPPO is that it turns sparse trajectory-level outcomes into dense supervision without requiring reference solutions or manually annotated intermediate steps. The completed validation results show that this signal can improve adapters across multiple model

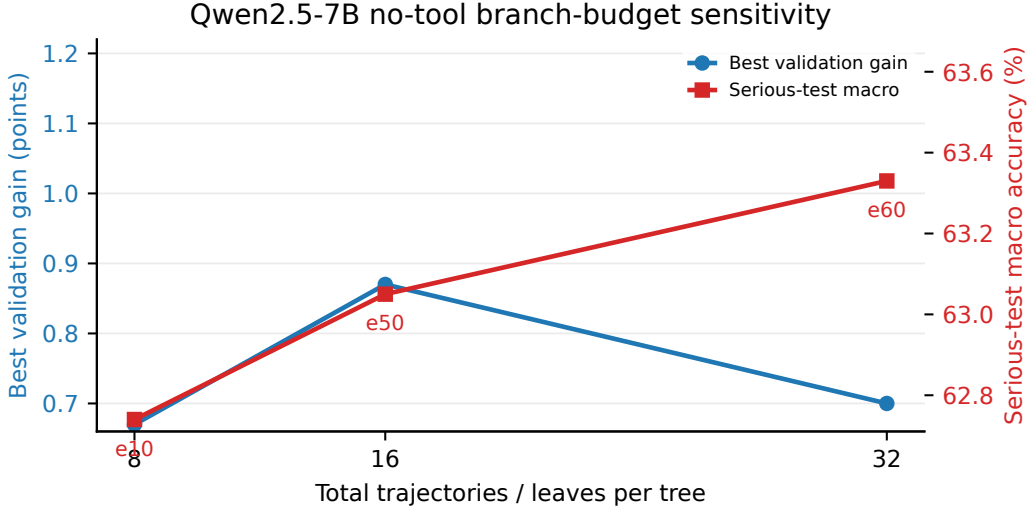


Figure 4: Branching-budget sensitivity for Qwen2.5-7B no-tool TreeMAPPO variants. The validation line reports the best validation improvement over each run’s epoch-0 checkpoint; the serious-test line reports the five-trial, six-dataset macro accuracy at the validation-selected checkpoint.

families, with gains comparable to matched GRPO under the low-learning-rate rank-32 configuration. The held-out serious tests are more mixed: TreeMAPPO is strongest in Qwen3-4B tool use and is positive for Gemma and Llama, but it does not beat the Qwen2.5 no-tool base model and underperforms on Mistral. The paper’s defensible conclusion is therefore not that tree-based credit assignment uniformly dominates GRPO, but that it is a viable dense-supervision mechanism whose benefit depends on model family, tool setting, and optimization stability.

Several practical design choices are worth emphasizing.

- Branch selection currently does not use posterior uncertainty; it uses structural penalties and the model’s alternative-token probability mass, which makes branching realistic and less likely to chase implausible continuations.
- The training loss uses a clipped policy-ratio surrogate with token-level advantages produced by the training-set generator. This keeps the optimization interface close to standard policy-gradient practice while preserving the central contribution: tree-based posterior credit assignment.
- Tool-enabled and no-tool settings are analyzed separately because the credit-assignment problem differs materially once trajectories can call Python and recover from arithmetic mistakes.
- The forced-token branch variant is now the default for guided branching because the branch score explicitly evaluates the probability of the selected alternative token; forcing that token makes the intervention match the scored decision.

9 LIMITATIONS AND FUTURE WORK

The present system has several important limitations.

- Tree construction requires more inference than flat rollout collection, so the method trades additional sampling cost for denser supervision.
- The main experiments use a split one-shot pipeline rather than fully online interleaved rollout and training. This is a practical choice driven by compute limits, queue latency, and debugging efficiency; it means the reported experiments primarily test the quality of the generated credit-assignment data, not the full closed-loop dynamics of repeated policy improvement.

- Current gains are modest and validation trends are noisy. The strongest completed validation gains range from below one percentage point in some Qwen2.5 and Gemma settings to roughly two to three points in Qwen3 tool use and Llama no-tool runs.
- The completed serious tests show mixed held-out transfer. This motivates future work on larger training sets, online policy refresh, stronger optimizer controls, and repeated independent seeds before making broad scaling claims.
- The MAP credit model intentionally uses a simple additive segment-contribution assumption. This makes inference tractable and interpretable, but it cannot represent all higher-order interactions among distant reasoning segments.
- Correctness is judged from extracted final answers, so the approach inherits the limitations of answer extraction and exact-answer comparison on mathematical benchmarks.
- The tool environment is deliberately lightweight. Richer agent tools may require additional safety checks, state tracking, and branch-selection criteria.

10 CONCLUSION

We presented TreeMAPPO, a trajectory-tree method for fine-grained credit assignment in agentic LLM training. The core idea is to branch reasoning trajectories at token granularity, infer segment-level contribution from mixed-success descendants, and convert those estimates into training signals for subsequent optimization. Completed split-pipeline experiments show that this signal can produce repeatable validation improvements and competitive held-out performance, with the strongest serious-test result appearing in the Qwen3-4B tool setting where TreeMAPPO outperforms matched GRPO. At the same time, Qwen2.5 no-tool and Mistral results show that tree-based credit assignment is not uniformly better than GRPO or the base model under the current offline pipeline. The defensible conclusion is that guided tree credit assignment is a viable and sometimes beneficial dense-supervision mechanism, but its gains depend on model family, tool setting, and optimization stability.

AI USE STATEMENT

Generative AI tools were used to assist with code implementation, experiment orchestration, log inspection, result aggregation, and manuscript editing. All reported results, claims, code changes, and paper text were reviewed by the authors, who take responsibility for the final content of this work.

ETHICS STATEMENT

This work studies mathematical reasoning benchmarks and does not involve human-subject data collection. The main ethical risks are ordinary risks of improving automated reasoning agents, including possible misuse and over-reliance on generated reasoning traces. We mitigate these risks by evaluating only answer correctness on public mathematical datasets and by reporting negative and mixed results alongside positive findings.

REPRODUCIBILITY STATEMENT

The paper reports the model families, rollout protocol, training hyperparameters, validation cadence, and testing protocol used in the experiments. The implementation uses chunked artifacts, explicit judging/scoring passes, and recorded metadata for validation and testing so that experiment outputs can be audited and regenerated.

A QUALITATIVE CASE STUDY

We inspected generated training trajectories for a Qwen2.5-7B tool-enabled TreeMAPPO run and selected a NuminaMath question that also appears in the no-tool ablation tree artifacts: “A woman is 42 years of age and her daughter is 8 years old. In a certain number of years, the mother will be

Tool prompt, exact decoded trajectory prefix.

```

<|im.start|>system
You are Qwen, created by Alibaba Cloud. You
are a helpful assistant.<|im.end|>
<|im.start|>user
You are a helpful agent that solves the
following problem.
Question: A woman is 42 years of age and
her daughter is 8 years old. In a certain
number of years, the mother will be three
times as old as her daughter. How many
years will it take for the mother to be
three times as old as her daughter?
You are encouraged to use the Python code
executor to help you do calculations to
ensure correctness. You can invoke python
code by emitting a Python markdown code
block like this:
```python
import numpy as np
A = np.array([[2, 1], [1, -1]])
b = np.array([5, 1])
solution = np.linalg.solve(A, b)
solution
```
You should reason step by step, and put the
final answer in a \boxed{ }.
Begin your reasoning:<|im.end|>
<|im.start|>assistant

```

No-tool prompt, exact decoded trajectory prefix.

```

<|im.start|>system
You are Qwen, created by Alibaba Cloud. You
are a helpful assistant.<|im.end|>
<|im.start|>user
You are a helpful agent that solves the
following problem.
Question: A woman is 42 years of age and
her daughter is 8 years old. In a certain
number of years, the mother will be three
times as old as her daughter. How many
years will it take for the mother to be
three times as old as her daughter?
You should reason step by step, and put the
final answer in a \boxed{ }.
Begin your reasoning:<|im.end|>
<|im.start|>assistant

```

Figure 5: Exact prompts for the same Qwen2.5-7B case-study question. These strings are decoded from the ignored prompt-token prefix in the msgpack training trajectories; they are not manually rewritten prompts.

three times as old as her daughter. How many years will it take for the mother to be three times as old as her daughter?" The correct equation is $42 + x = 3(8 + x)$, giving $x = 9$.

Figure 5 gives the exact decoded prompts seen by the model in the tool and no-tool TreeMAPPO settings, including the Qwen chat-template markers present in the trajectory artifacts. Figures 6 and 7 show exact raw trajectory excerpts from TreeMAPPO and available ablations. For the main tool/no-tool comparison, we use the same question. For the ablation panels, we intentionally allow different questions because questions whose sampled leaves are uniformly correct or uniformly incorrect produce zero or near-zero advantages under several credit rules; such cases are uninformative for visualizing how each method distributes credit. The displayed ablation questions are listed in each panel.

The colors are used only to visualize token-level training advantages for readers and automated reviewers; they are not part of the model input or output. Prompt tokens and Python tool outputs have ignored labels and are not colored. Each trajectory uses an adaptive muted red–yellow–green interpolation: the largest absolute advantage in that trajectory saturates to dark red or dark green, so color shows within-trajectory credit structure while the printed numbers show cross-trajectory scale. In the tool-enabled TreeMAPPO trajectory, the model first solves the equation with a Python call but writes an incorrect answer, receives the actual tool response 9, and then corrects the final answer to 9. The early setup receives weak negative credit, while the post-tool repair receives positive credit.

This appendix is qualitative rather than a substitute for aggregate validation. It illustrates why the TreeMAPPO design combines guided branching with posterior segment credit. GRPO supplies only a whole-response terminal signal, so it cannot distinguish setup, computation, and repair inside a response. TreeRPO- and TreeRL-style backups can create useful sign changes, but their values are coarser because they are backed up from subtree correctness rates rather than inferred as latent segment contributions. TEMPO-style branching and TreeRL-style entropy branching can expose alternative paths, but their branch construction is less tightly coupled to the token-level intervention used by TreeMAPPO. In the tool-enabled TreeMAPPO trajectory, the method preserves the mistaken pre-tool context, observes the tool response, and assigns positive credit to the later repair segment without treating the entire response as uniformly good.

Question. A woman is 42 years old and her daughter is 8 years old. In how many years will the mother be three times as old as her daughter?

Colored raw trajectory excerpt. Let's denote the number of years from now when the mother will be three times as old as her daughter by (x) . Currently, the mother is 42 years old and the daughter is 8 years old. In (x) years, the ages are $(42+x)$ and $(8+x)$. According to the problem, the mother's age will be three times the daughter's age: $[42+x=3(8+x)]$. Now we solve this equation using Python: `python from sympy import symbols, Eq, solve ... solution [0] '' :boxed{6}`. So, it will take 6 years for the mother to be three times as old as her daughter.

`<tool.response>9</tool.response>`

It seems there was an error in my previous calculation. Let's re-solve the equation step by step: $[42+x=3(8+x)]$, so $[42+x=24+3x]$. Rearranging terms, $[42-24=3x-x]$, $[18=2x]$, and $[x=9]$. Therefore, the correct answer is $[boxed{9}]$.

Observed segment advantages. $-0.114, +0.222, +0.635, +0.635, +0.576$.

Figure 6: Case study from Qwen2.5-7B with tool use. The displayed text is copied from the msgpack training trajectory, with only LaTeX escaping and ellipses added for space. The black tool-response line is ignored by the training loss but explains why the following model segment can repair the answer.

REFERENCES

- Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *arXiv preprint arXiv:2211.12588*, 2022. URL <https://arxiv.org/abs/2211.12588>.
- Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. Pal: Program-aided language models. *arXiv preprint arXiv:2211.10435*, 2022. URL <https://arxiv.org/abs/2211.10435>.
- Zhiwei He, Tian Liang, Jiahao Xu, Qiuzhi Liu, Xingyu Chen, Yue Wang, Linfeng Song, Dian Yu, Zhenwen Liang, Wenxuan Wang, Zhuosheng Zhang, Rui Wang, Zhaopeng Tu, Haitao Mi, and Dong Yu. Deepmath-103k: A large-scale, challenging, decontaminated, and verifiable mathematical dataset for advancing reasoning. *arXiv preprint arXiv:2504.11456*, 2025. URL <https://arxiv.org/abs/2504.11456>.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *Advances in Neural Information Processing Systems*, 2021. URL <https://arxiv.org/abs/2103.03874>.
- Bingning Huang, Tu Nguyen, and Matthieu Zimmer. Tree-opo: Off-policy monte carlo tree-guided advantage optimization for multistep reasoning, 2025. *arXiv preprint*.
- Yizhi Li, Qingshui Gu, Zhoufutu Wen, Ziniu Li, Tianshun Xing, Shuyue Guo, Tianyu Zheng, Xin Zhou, Xingwei Qu, Wangchunshu Zhou, Zheng Zhang, Wei Shen, Qian Liu, Chenghua Lin, Jian Yang, Ge Zhang, and Wenhao Huang. Treepo: Bridging the gap of policy optimization and efficacy and inference efficiency with heuristic tree-based modeling, 2025.
- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. *arXiv preprint arXiv:2305.20050*, 2023. URL <https://arxiv.org/abs/2305.20050>.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *arXiv preprint arXiv:2302.04761*, 2023. URL <https://arxiv.org/abs/2302.04761>.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017.

- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024. URL <https://arxiv.org/abs/2402.03300>.
- Hieu Tran, Zonghai Yao, and Hong Yu. Exploiting tree structure for credit assignment in rl training of llms, 2025. TEMPO.
- Xucong Wang, Ziyu Ma, Yong Wang, Yuxiang Ji, Shidong Yang, Guanhua Chen, Pengkun Wang, and Xiangxiang Chu. Appo: Agentic procedural policy optimization, 2026.
- Zhicheng Yang, Zhijiang Guo, Yinya Huang, Xiaodan Liang, Yiwei Wang, and Jing Tang. Treerpo: Tree relative policy optimization, 2025.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022. URL <https://arxiv.org/abs/2210.03629>.

TreeMAPPO, no-tool. *Question:* A woman is 42 years old and her daughter is 8 years old. In how many years will the mother be three times as old as her daughter?
The sampled no-tool tree solves the age equation directly, so the posterior has little corrective contrast to exploit.
Let's denote the number of years from now when the mother will be three times as old as her daughter as x . Currently, the mother's age is 42 years and the daughter's age is 8 years. In x years, the mother's age will be $42+x$ and the daughter's age will be $8+x$. We can set up the equation $42+x=3(8+x)$, solve it, and obtain $x=9$. The final answer is $\boxed{9}$.
Advantages: +0.074, 0.

GRPO, no-tool. *Question:* Find the auxiliary equation for the ordinary differential equation with constant coefficients $(x^2 D^2 + xD + 1)y = \sin(2 \log x) \sin(\log x)$.
GRPO assigns one terminal group-normalized advantage to the entire response.
To find the auxiliary equation for the given ordinary differential equation with constant coefficients, we first recognize that the equation is not in a standard constant-coefficient form because the operator contains $(x^2 D^2 + xD + 1)$. We rewrite it through the substitution $(t = \log x)$, where (xD) acts like differentiation with respect to (t) . This yields the auxiliary equation $(m^2 + 1 = 0)$.
Advantages: +1.732 for this representative response.

TEMPO-style branching. *Question:* Find the auxiliary equation for the ordinary differential equation with constant coefficients $(x^2 D^2 + xD + 1)y = \sin(2 \log x) \sin(\log x)$.
Spontaneous prefix-tree branching produces a nonzero but fragmented early credit pattern.
To find the auxiliary equation for the given ordinary differential equation, we need to follow these steps. First identify the homogeneous part of the ODE: $((x^2 D^2 + xD + 1)y = 0)$. Assume $(y = x^m)$, substitute $(y' = mx^{m-1})$ and $(y'' = m(m-1)x^{m-2})$, and simplify to $(m^2 + 1 = 0)$.
Advantages: +0.162, +0.111, -0.263, +0.217, +0.633.

TreeRPO-style credit. *Question:* Suppose $g(x) = 5x - 3$. What is $g^{-1}(g^{-1}(14))$?
The child-group win-rate rule gives a strong sign change around the transition from generic inverse derivation to the nested evaluation.
To solve for $(g^{-1}(g^{-1}(14)))$, first find the inverse of $(g(x) = 5x - 3)$. Set $(y = 5x - 3)$, solve $(x = (y + 3) / 5)$, and obtain $(g^{-1}(x) = (x + 3) / 5)$. Then compute $(g^{-1}(14) = 17/5)$, followed by $(g^{-1}(17/5) = ((17/5) + 3) / 5 = 32/25)$. Therefore, the final answer is $\boxed{32/25}$.
Advantages: -0.525, +2.000.

TreeRL-style credit. *Question:* Given $f(x, y) = e^{-(x+y)}$ for $x > 0, y > 0$, compute $P(Y > X + 1)$.
The local-global value rule creates a coarse negative-to-positive split over the integral setup and evaluation.
To compute $(P(Y > X + 1))$, integrate $(e^{-(x+y)})$ over $(x > 0)$ and $(y > x + 1)$. Set up $(\int_0^\infty \int_{x+1}^\infty e^{-(x+y)} dy dx)$. The inner integral becomes $(\int_{2x+1}^\infty e^{-u} du = e^{-(2x+1)})$, after which the remaining integral can be evaluated.
Advantages: -0.219, +0.281.

TreeRL-style branching. *Question:* Evaluate $\lim_{x \rightarrow 0} (1/\tan^2 x - 1/x^2)$.
Entropy-guided branching yields a long recovery pattern with a negative initial transformation and a positive Taylor-expansion segment.
Start by writing $(\tan x = \sin x / \cos x)$, so $(1/\tan^2 x = \cos^2 x / \sin^2 x)$. Combining terms gives $((x^2 \cos^2 x - \sin^2 x) / (x^2 \sin^2 x))$. Then analyze the numerator and denominator as $(x \rightarrow 0)$ using Taylor series for $(\sin x)$ and $(\cos x)$, which reveals the limiting constant after cancellation.
Advantages: -0.323, +0.805.

Figure 7: Representative trajectory excerpts for available Qwen2.5-7B no-tool ablations. Unlike the main tool/no-tool comparison, ablation panels may use different questions so that the visualized examples avoid the uniform-correctness regime where several advantage rules collapse to zero. The examples are reconstructed from judged tree artifacts for qualitative analysis.